

Database Model Recommendation.

Recommendation and Justification.

The model adopted for vehicle dealership is Relational Database Management System (RDBMS) for the following reasons:

Organized and Clear-cut Data: The main processes of the dealership, namely sales and service, are dependent on very well-organized and clearly related data. Such objects as Vehicles, Customers, Sales, Service_Orders, and Technicians have attributes of a predetermined nature (e.g., VIN, Customer ID, Sale Date) that nest well into the rows and columns of relational tables.

Complicated Relationships: The business logic has to deal with complicated relationships (one-to-many, many-to-many). For example, A single Dealership Location is capable of having a large number of Sales Staff. A Service_Order may entail a number of Services being done by many Technicians. The RDBMS is very effective in managing these relationships by the use of primary and foreign keys to guarantee the Referential Integrity. This eliminates orphaned records e.g. a sale of a nonexistent vehicle.

Data Integrity and ACID Compliance: Sales prices, costs of services, status of vehicles in the inventory are the issues which need to be absolutely accurate and consistent. The ACID properties (Atomicity, Consistency, Isolation, Durability) of the relational databases ensure that all the transactions are done with consistency which is essential in the financial and operational reporting of the business.

Complex querying with SQL: The demanding nature of complex reporting, whether it is the highest performing salesperson by location, summed sales revenue or individualized technician certification, is best handled by the well developed and robust SQL. SQL is formulated by complex queries that make use of joins, aggregations, and filtering of a series of related tables.

Future possible inefficiencies and Recommendations.

Although the relational model is optimal at the moment, some of the situations that might occur in future may expose inefficiencies:

1- In Processing Unstructured or semi-structured Data:

Use case: The dealership decides to expand its functions and add customer feedback analysis on the social media, sensor information on connected cars to predict maintenance, or even pictures of vehicle damage.

Inefficiency: Relational databases have not been created to effectively store and retrieve large amounts of unstructured data. It would be difficult and slow to put this data into shrink-wrapped table formats.

Alternative Recommendation: A change to Multi-Model Database or a NoSQL database (such as MongoDB to store documents or Cassandra to store time-series data) in addition to the RDBMS. This forms a polyglot persistence architecture in which different databases are utilised to store the type of data that they best represent.

2- Extreme Scalability Conditions of Basic Operations:

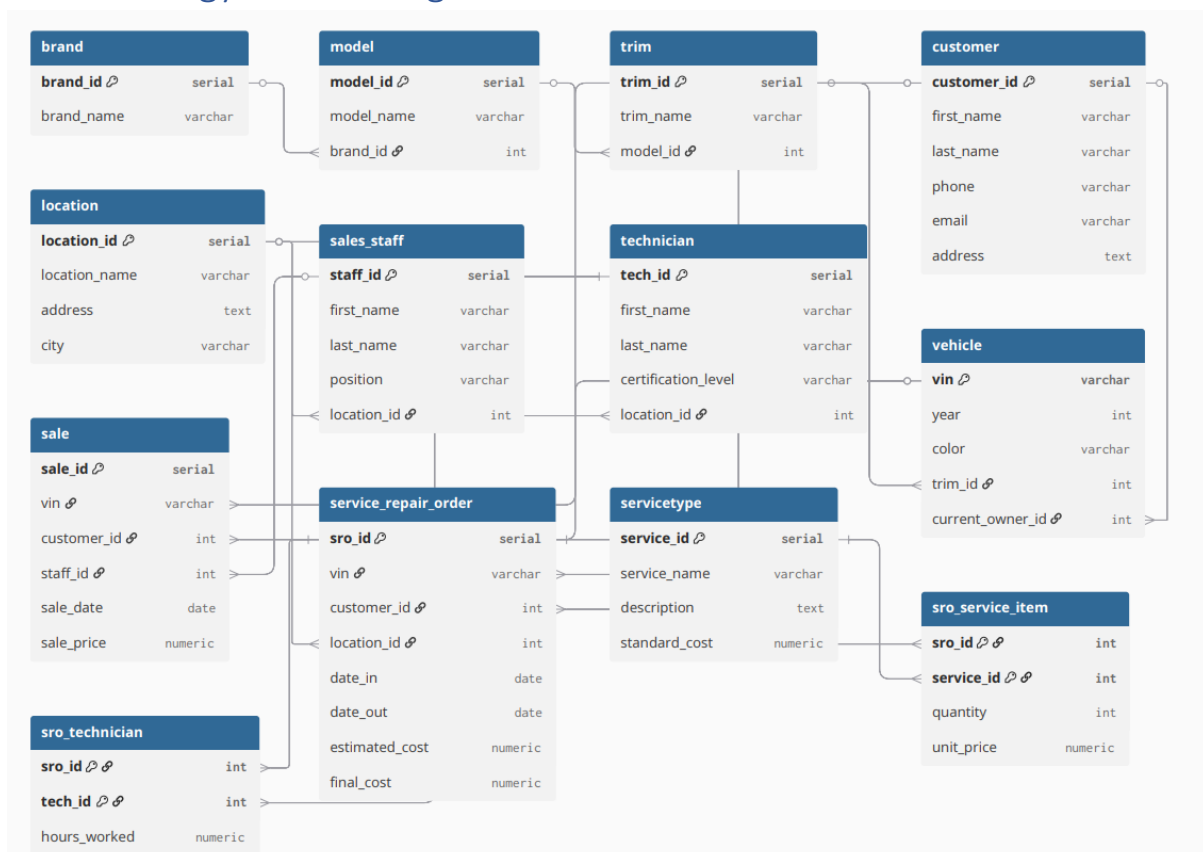
Scenario: The company becomes a national franchise and has hundreds of branches, producing millions of simple read/write operations per second (e.g. high-frequency data of the IoT in thousands of cars, real-time monitoring of inventory across the country).

Inefficiency: RDBMSs are scaleable, but vertical scaling (adding more power to one server) can be expensive and horizontal scaling (sharding) is an expensive and complicated concept that can violate the ACID properties.

Alternative Recommendation: In these particular, high volume and low complexity data streams, a NoSQL database such as a Key-Store or Wide-Column Store can be incorporated. These systems are made to be massively horizontally scalable and highly available, at the expense of part of the consistency guarantees (conformity to the BASE model).

To sum up, Relational Model is the appropriate and most effective model that can be adopted by the dealership with the existing structured and complicated relationship data. That the architecture should however be built to scale to future needs, to be willing to have complementary solutions to certain, excessive, or un-structured data needs appear that might arise as the business grows.

Methodology for creating Schema



The processes that take place in formulating the relational schema (which is equally echoed by the ERD) based on the business needs of sales and service management are as follows.

Schema Design Procedure (Step-by-Step).

It is a design that is grounded on the principles of relational modeling the design process is initiated by identification of high-level entities and then proceeds to the determination of detailed attributes and constraints.

Step 1: Entity Identification

The first one is to analyse the business requirements and identify the important objects (entities) that are to be tracked.

Basic Business Entities Customer, Vehicle, Sale and Service RepairOrder (SRO).

Support/Lookup Entities: Brand, Model, Location, Technician and SalesStaff.

Step 2: Attribute Definition

The values needed by any entity are identified (data fields). It implies that all the required data of the requirements (VIN, price, date, technician certification, contact details of the customer etc) must be considered.

Sample: the following attributes would be needed in the CUSTOMER entity; lastname, firstname, phone, email and address.

Step 3: Choice of Primary Key (PK).

Every row of a table is assigned an identifier which is known as a Primary Key (PK). Surrogate keys (id) are created in situations where natural keys are absent or inconvenient to utilize.

Natural PK: VEHICLE uses vin.

Surrogate PKs: customerid- CUSTOMER, saleid- saleid, brandid- BRAND.

Step 4 Relationship Identification and Foreign Key (FK).

The entities are organized in accordance with business rules (e.g. a Customer owns a Vehicle, a Sale includes a single Vehicle). These relationships are implemented by the aid of Foreign Keys (FKs) the PKs of one table that is introduced to another one.

Examples: SALE should include a reference to VEHICLE and CUSTOMER therefore it is composed of a vin (FK) and customerid (FK).

Complex Relationship: This is a many-to-many relationship between SRO and SERVICE that is resolved by the establishment of an associative table (SROSERVICEITEM) with sroid (FK) and serviceid (FK).

Step 5: Normalisation (Note 3NF)

The design is checked against the normalization rules (typically to the Third Normal Form - 3NF) to minimize redundancy of data and the occurrence of the insertion/update anomalies.

Example: Technician details (firstname, certification) are moved to a separate TECHNICIAN table in order to avoid the repetition of these details by a separate SRO to which a technician is assigned. The two tables are then associated with SROTECHNICIAN.

Note: SROTECHNICIAN the first ERD has also firstname, lastname and certification. It implies that it can be somehow reliant on it and will have to be removed in the final design since the only way to join to the TECHNICIAN table is the techid (FK) and hence achieve a 3NF.

Step 6: Constraint and Rule Definition.

Finally, some constraints are also included to ensure quality of data. This includes NOT NULL restraints of the fields, which are required and Unique Constraints to prevent the entries of the duplicates where it is needed.

Example: The SALE table will have a distinct constraint on the vin to eliminate the selling of the vehicle multiple times on the sales record.